Where is work waiting, and what would actually improve the flow?

Student notes for lecture 4. One small team improves Campus Rooms with eight work cards on one board. Each scene shows a situation, asks you to predict, reveals the mechanism, changes one condition, compares the outcomes and names the principle: feedback batches, coordination approaches, a bottleneck, lead and cycle time, estimates, risk-driven decisions, a completion policy and a process change tested like a hypothesis.

Scenes in this lecture

#	SCENE (STUDENT HEADING)	ROLE	DEEP LINK
01	When would the team learn it built the wrong thing?	Anchor	`week-04.html#/feedback-batches`
02	Does a meeting create a feedback loop by itself?	Bridge	`week-04.html#/methods-not-rituals`
03	Make one stage faster: does the work finish sooner?	Anchor	`week-04.html#/queue-and-bottleneck`
04	Which clock does the person who asked experience?	Bridge	`week-04.html#/wip-and-time`
05	“Three days”: effort, elapsed time or a promise?	Bridge	`week-04.html#/estimates-with-assumptions`
06	What should the team learn before committing more work?	Anchor	`week-04.html#/risk-and-learning`
07	The code runs. Is the cancellation change done?	Bridge	`week-04.html#/definition-of-done`
08	The review queue keeps growing. What should change?	Bridge	`week-04.html#/retrospective-as-test`

Campus Rooms is a fictional room-booking service; every card, round, count and example value is a teaching example unless a source is named. The board is a deliberately simplified queue model with equal-sized cards: its rounds are teaching units, not a forecast of anyone's productivity. The method comparison is simplified; the Scrum Guide (S04) and the Kanban Guide (S05) define each method.

Everyone is busy. Why are only three cards done?

A small team is improving Campus Rooms. Eight work cards, one board, four rounds of work so far. Before changing anything, look at where the work is.

The situation: the board after round 4

Ready: none. Waiting for review: W04, W05, W06, W07, W08. Done: W01, W02, W03. Implementation takes 2 cards per round, review 1.
Room staff ask when W05, accessible feedback, will be ready. Started: 8 of 8; done: 3. 5 cards wait for review, W05 among them.

The question of this week

Where is work waiting, and what would actually improve the flow? (A) Make implementation faster: it looks busiest. (B) Find where the cards wait, and improve that step. (C) Start more cards, so nobody is idle.

Four questions this lecture answers

- When does the team learn it built the wrong thing? (scenes 1–2)
- Where is the work waiting, and for how long? (scenes 3–4)
- What does “three days” promise, and what should come first? (scenes 5–6)
- When is a change done, and did a process change help? (scenes 7–8)

What this lecture builds

One board of eight cards, run round by round, and the decisions made around it.

What you already know, and where this lecture goes

Where is work waiting, and what would actually improve the flow?

From Week 3

Five models of B-001: a journey, two state models, a data model, a decision table and sequence diagrams. Week 3 ended on the question: Who turns these models into working behavior, in what order, and where does that work wait?

The example this week: eight work cards

- W01 conflict rule
- W02 duration boundary
- W03 cancellation ownership
- W04 room closure
- W05 accessible feedback
- W06 notification recovery
- W07 search response time
- W08 support diagnosis

How the board works

Time moves in rounds. Implementation takes 2 cards per round, review 1; a card implemented in a round joins the review queue for the following round. Every card is the same size and all eight are requested at round 0. Eight equal-sized cards arrive at round0. Review consumes only cards available at round start. Completed implementation transfers for the following round. Counts describe end-of-round state. Rounds are teaching units, not measured developer performance.

By the end of this lecture you can

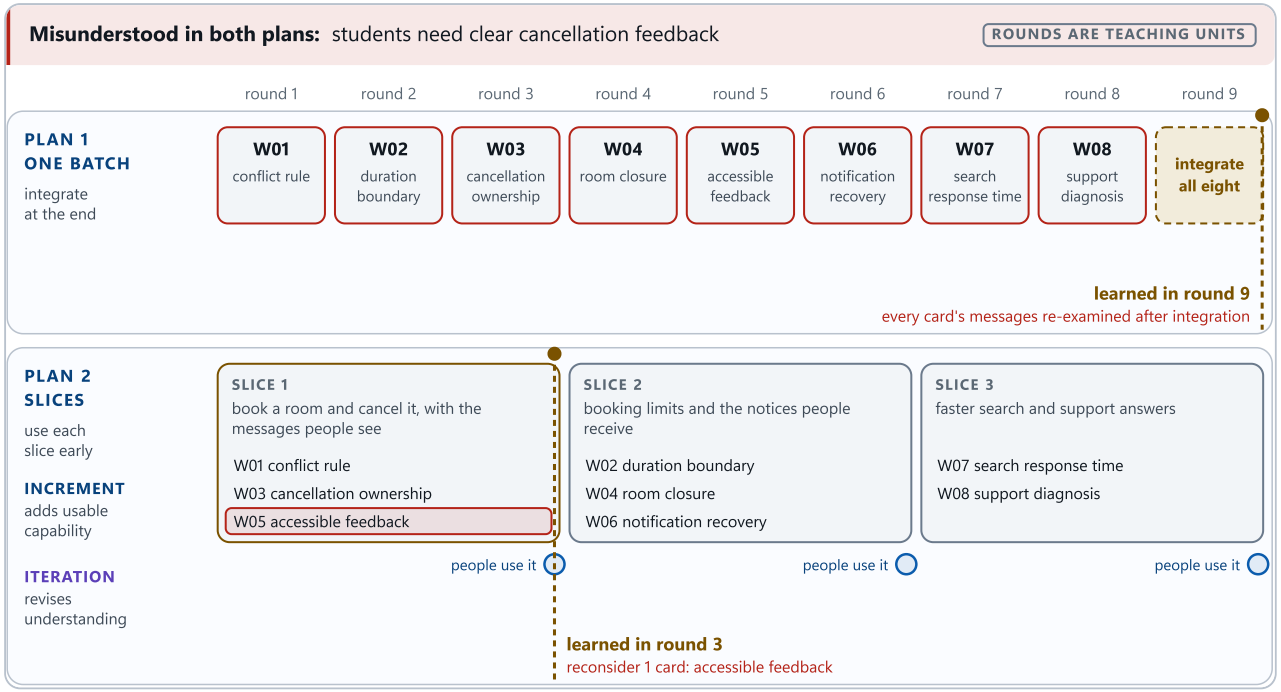
1. Distinguish iteration, increments, and a single large batch of delivery.
2. Locate a bottleneck using queues and completion evidence.
3. Distinguish work-in-progress, throughput, cycle time, and lead time.
4. Explain an estimate as conditional information rather than a guarantee.
5. Choose a next action using risk, dependency, feedback value, and capacity.

When would the team learn it built the wrong thing?

Panel 1 of 3 · One batch and three slices: when the misunderstanding shows, and what must change

Condition: **Baseline: one batch, or small slices** State: **7 of 7**

Learned in round 3 instead of round 9.



Assumptions: eight equal-sized cards, one team, the same misunderstanding in both plans (students need clear cancellation feedback); rounds are teaching units. Plan 1 builds one card per round and integrates in round 9; the misunderstanding shows in round 9, and the rework is every card's messages re-examined after integration. Plan 2 lets people use slice 1 (conflict rule, cancellation ownership and accessible feedback) in round 3; only accessible feedback changes. An increment adds usable capability; an iteration revises understanding.

Principle · Shorter feedback loops can expose wrong assumptions earlier; useful increments must still make sense as a system.

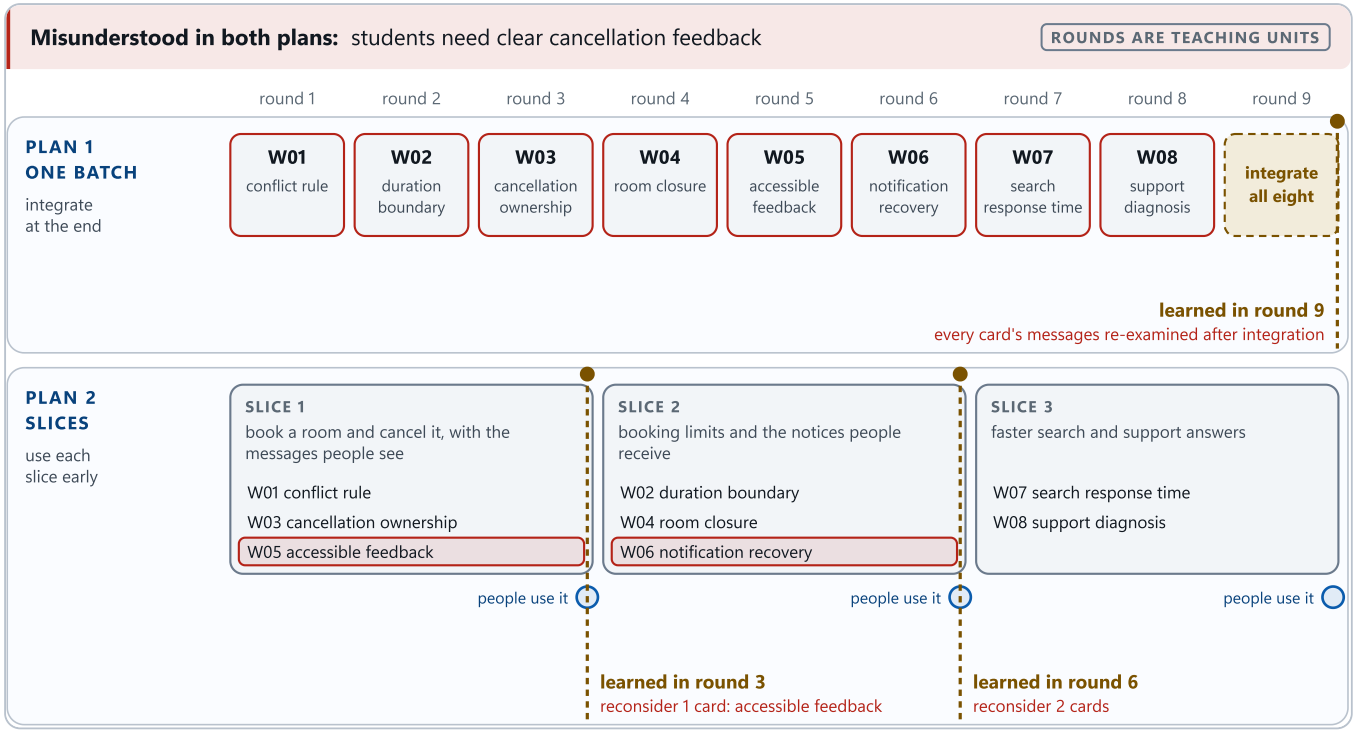
In short · Show slices early; make them fit.

When would the team learn it built the wrong thing?

Panel 2 of 3 · The feedback point moves later: what must be reconsidered

Condition: **Changed: the feedback point moves later** State: **5 of 5**

1, then 2, then 8 cards to redo.



Feedback in round 3: the first slice is in use; only its cancellation message changes; reconsider accessible feedback. Feedback in round 6: the second slice reused the unclear cancellation message in its notices; reconsider accessible feedback and notification recovery. Feedback in round 9: every card's messages re-examined after integration; reconsider all 8 cards.

Principle · Shorter feedback loops can expose wrong assumptions earlier; useful increments must still make sense as a system.

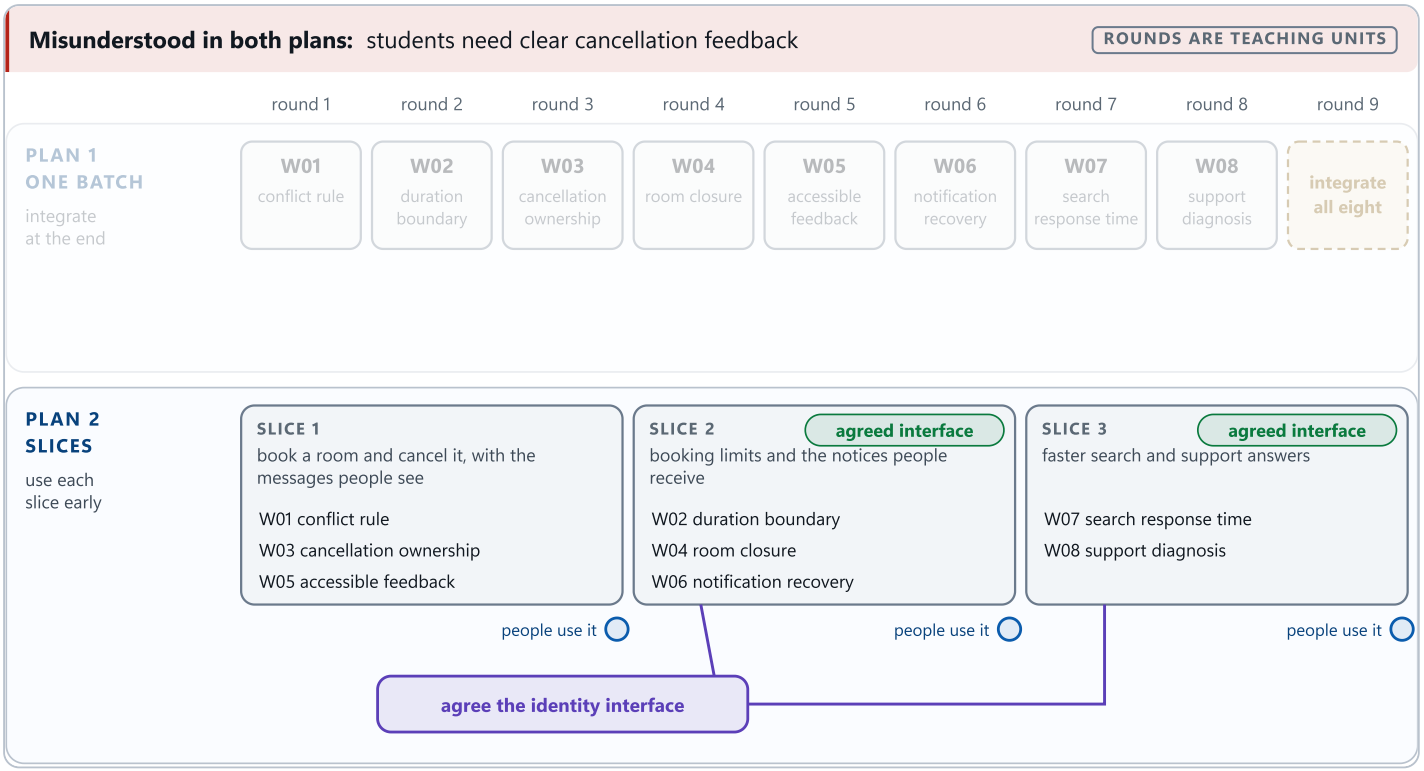
In short · Show slices early; make them fit.

When would the team learn it built the wrong thing?

Panel 3 of 3 · Small slices still need an agreed interface and an order

Condition: **Changed: slices share one identity interface** State: **4 of 4**

Agree it by round 3, then build on it.



The identity interface must be agreed before slices two and three. Without that agreement, slices two and three each assume their own way of naming a member, and joining them reopens both. With it agreed by round 3, slices 2 and 3 are built on one interface.

Principle · Shorter feedback loops can expose wrong assumptions earlier; useful increments must still make sense as a system.

In short · Show slices early; make them fit.

Does a meeting create a feedback loop by itself?

Panel 1 of 2 · Three ways to coordinate one team, compared row by row

Condition: **Baseline: a planned capability** State: **7 of 7**

A meeting without something usable is ritual.

	Plan-driven a plan agreed up front	**Iterative timeboxes (Scrum)** a goal for each Sprint	**Flow policies (Kanban)** a visible workflow with explicit policies
COORDINATES THROUGH	A plan agreed up front; each stage hands its result to the next	A Sprint Goal for a Sprint of one month or less	A defined workflow with explicit policies and a controlled number of items in progress
MAKES VISIBLE	The plan, the milestones and each stage handoff	The Product Backlog, the Sprint Goal and the Increment	Where each item is, how long it has aged, and where work waits
FEEDBACK ARRIVES	At planned milestones and handoffs	The Sprint Review inspects the Increment; the Retrospective inspects how the team works	Continuously, from flow measures and delivered items
STAYS OPEN TO CHANGE	Little once the handoffs have started	The next Sprint Goal and the order of the Product Backlog	Which item to pull next, and the workflow policies themselves
STRONGEST WHEN	Costly external integration, or a required evidence handoff	A product goal whose details are learned from each usable Increment	A steady stream of items whose arrival times are unknown

SCRUM ACCOUNTABILITIES

Product Owner · value | Scrum Master · process | Developers · the work

Accountabilities, not job titles: where decisions about value, process and the work become visible.

LEARN BEFORE COMMITTING

Try an early prototype, or a risk investigation: both work in any of the three.

A MEETING ALONE IS NOT FEEDBACK

For example a review with nothing usable to inspect, or a board nobody pulls from.

These are simplified lecture examples, not complete method definitions: the Scrum Guide (November 2020) and the Kanban Guide (May 2025) define each method. Scrum's three accountabilities, Product Owner (value: what the product needs next), Scrum Master (process: how well Scrum works for the team), Developers (the work: how to build a usable Increment), are accountabilities, not job titles. Scrum rests on transparency, inspection and adaptation; the Kanban Guide's practices are defining and visualizing a workflow, actively managing items in a workflow and improving a workflow. Another way to learn before committing: an early prototype and a risk investigation.

Principle · Choose coordination practices for the uncertainty and constraints of the work, not for their labels.

In short · Pick practices for the work, not the name.

Does a meeting create a feedback loop by itself?

Panel 2 of 2 · The work changes to incidents that arrive at unknown times

Condition: **Changed: incidents arrive at unknown times** State: **4 of 4**

Every approach needed a new policy.

	Plan-driven a plan agreed up front	**Iterative timeboxes (Scrum)** a goal for each Sprint	**Flow policies (Kanban)** a visible workflow with explicit policies
STILL HELPS	The fixed parts: an agreed date for the external integration	The Sprint Goal keeps the planned work visible	Arrivals join a visible queue; the limit on items in progress protects the work already started
NEEDS A NEW POLICY	No milestone can schedule an arrival nobody knows yet	A policy for urgent work: how much capacity it may take, and when the Product Owner renegotiates the goal	An explicit policy for urgent items and a limit on how many run at once

NO UNIVERSAL WINNER

Each approach needs a revised policy; none of them is the universal answer. The question is what the work needs: when feedback arrives, and which decisions can still change.

Support reports arrive at unknown times, some urgent. Plan-driven: still helps with the fixed parts: an agreed date for the external integration; needs no milestone can schedule an arrival nobody knows yet. Iterative timeboxes (Scrum): still helps with the Sprint Goal keeps the planned work visible; needs a policy for urgent work: how much capacity it may take, and when the Product Owner renegotiates the goal. Flow policies (Kanban): still helps with arrivals join a visible queue; the limit on items in progress protects the work already started; needs an explicit policy for urgent items and a limit on how many run at once. Each approach needs a revised policy; none of them is the universal answer.

Principle · Choose coordination practices for the uncertainty and constraints of the work, not for their labels.

In short · Pick practices for the work, not the name.

Make one stage faster: does the work finish sooner?

Panel 1 of 4 · Round 4 of the baseline: every card started, the review queue at its peak

Condition: **Baseline: implementation 2, review 1 per round** State: **5 of 10**

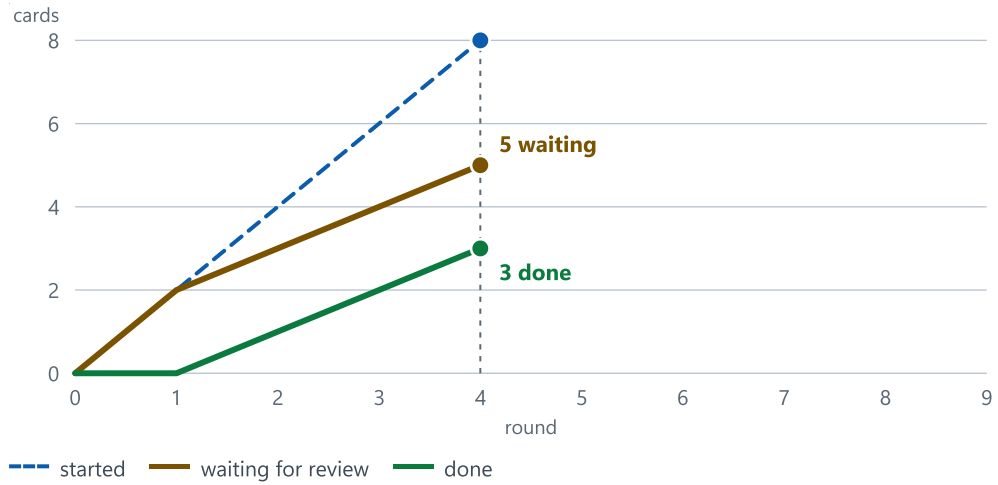
Round 4: all eight cards are started and only **3** are done. **5** cards wait for review, and both stations were busy.

READY	IMPLEMENTING	WAITING FOR REVIEW	REVIEW	DONE
0	2 per round this round W07 W08	5 W04 W05 W06 W07 W08	1 per round this round W03	3 W01 W02 W03

The cards: W01 conflict rule · W02 duration boundary · W03 cancellation ownership · W04 room closure · W05 accessible feedback · W06 notification recovery · W07 search response time · W08 support diagnosis

QUEUE AND COMPLETION BY ROUND

Started 8 · waiting 5 · done 3



END OF EACH ROUND

ROUND	READY	WAITING	DONE
0	8	0	0
1	6	2	0
2	4	3	1
3	2	4	2
4	0	5	3

Capacities: Implementation 2 per round, Review 1 per round. Transfer rule: a card implemented in a round joins the review queue for the following round; review takes only cards waiting at the start of a round. Counts are end-of-round. Started 8, waiting 5, done 3.

Make one stage faster: does the work finish sooner?

Panel 2 of 4 · The baseline to the end: review sets the pace

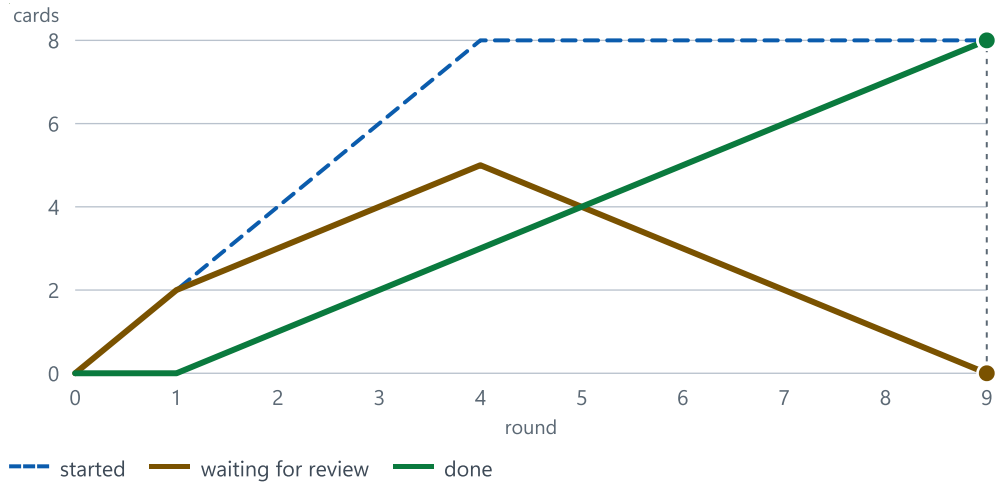
Condition: **Baseline: implementation 2, review 1 per round** State: **10 of 10**

Round 9: **W08** is done. Implementation finished its part in round 4, the board in **round 9**: review set the pace.

READY	IMPLEMENTING	WAITING FOR REVIEW	REVIEW	DONE
0	2 per round	0	1 per round	8
	this round		this round	W01 W02 W03 W04
	idle idle		W08	W05 W06 W07 W08

The cards: W01 conflict rule · W02 duration boundary · W03 cancellation ownership · W04 room closure · W05 accessible feedback · W06 notification recovery · W07 search response time · W08 support diagnosis

QUEUE AND COMPLETION BY ROUND — **Started 8 · waiting 0 · done 8**



END OF EACH ROUND

ROUND	READY	WAITING	DONE
0	8	0	0
1	6	2	0
2	4	3	1
3	2	4	2
4	0	5	3
5	0	4	4
6	0	3	5
7	0	2	6
8	0	1	7
9	0	0	8

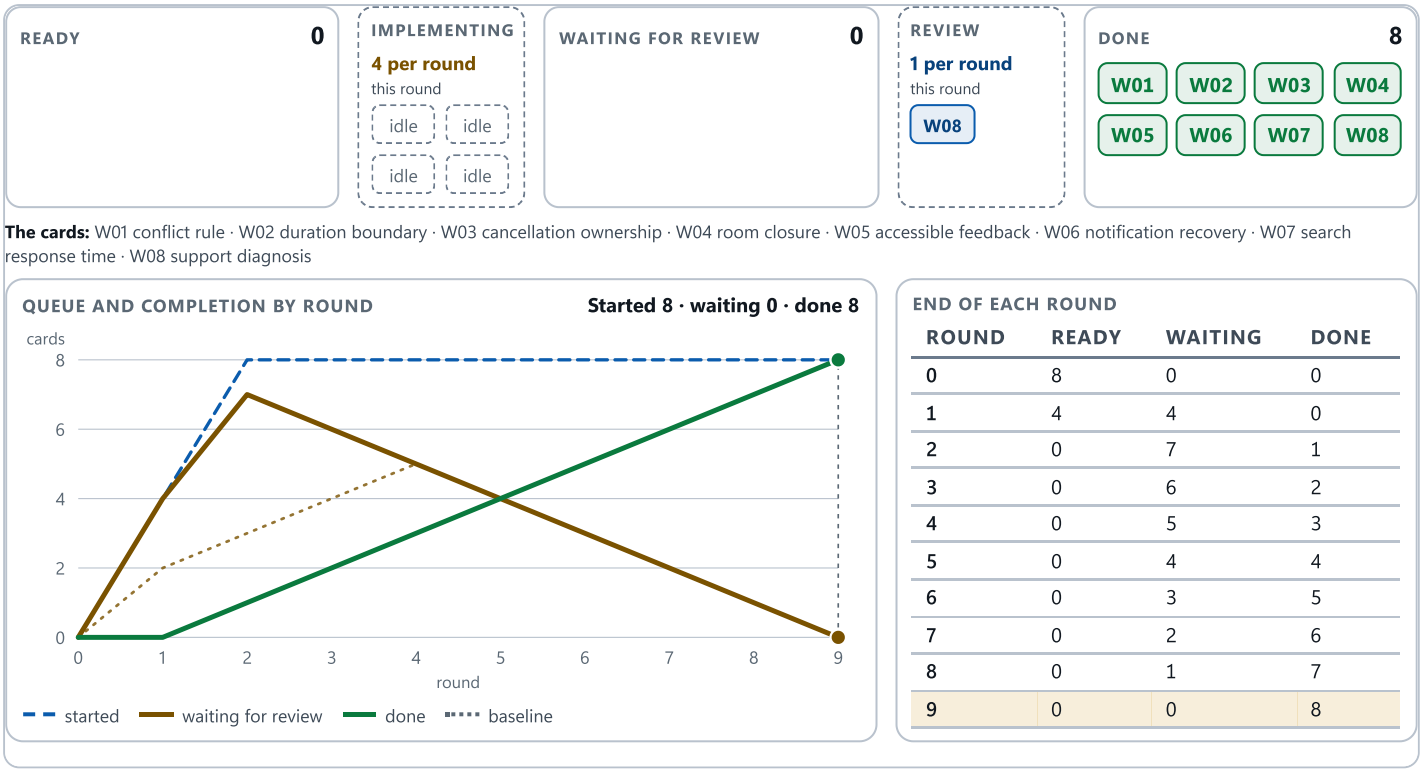
Implementation 2 per round, Review 1 per round. All eight cards done in round 9; the longest review queue was 5 cards (round 4). rounds are teaching units, not measured developer performance.

Make one stage faster: does the work finish sooner?

Panel 3 of 4 · Implementation doubled: the same finish, a longer queue

Condition: **Changed: implementation 4, review 1 per round** State: **10 of 10**

Done in round 9, as before.



END OF EACH ROUND

ROUND	READY	WAITING	DONE
0	8	0	0
1	4	4	0
2	0	7	1
3	0	6	2
4	0	5	3
5	0	4	4
6	0	3	5
7	0	2	6
8	0	1	7
9	0	0	8

Implementation 4 per round, Review 1 per round, the same arrivals. Done in round 9, as in the baseline; the review queue peaks at 7 instead of 5. Doubling implementation did not change completion, because review still finishes 1 card per round.

Principle · Local activity is not the same as system throughput; improve the limiting flow, not merely the busiest-looking stage.

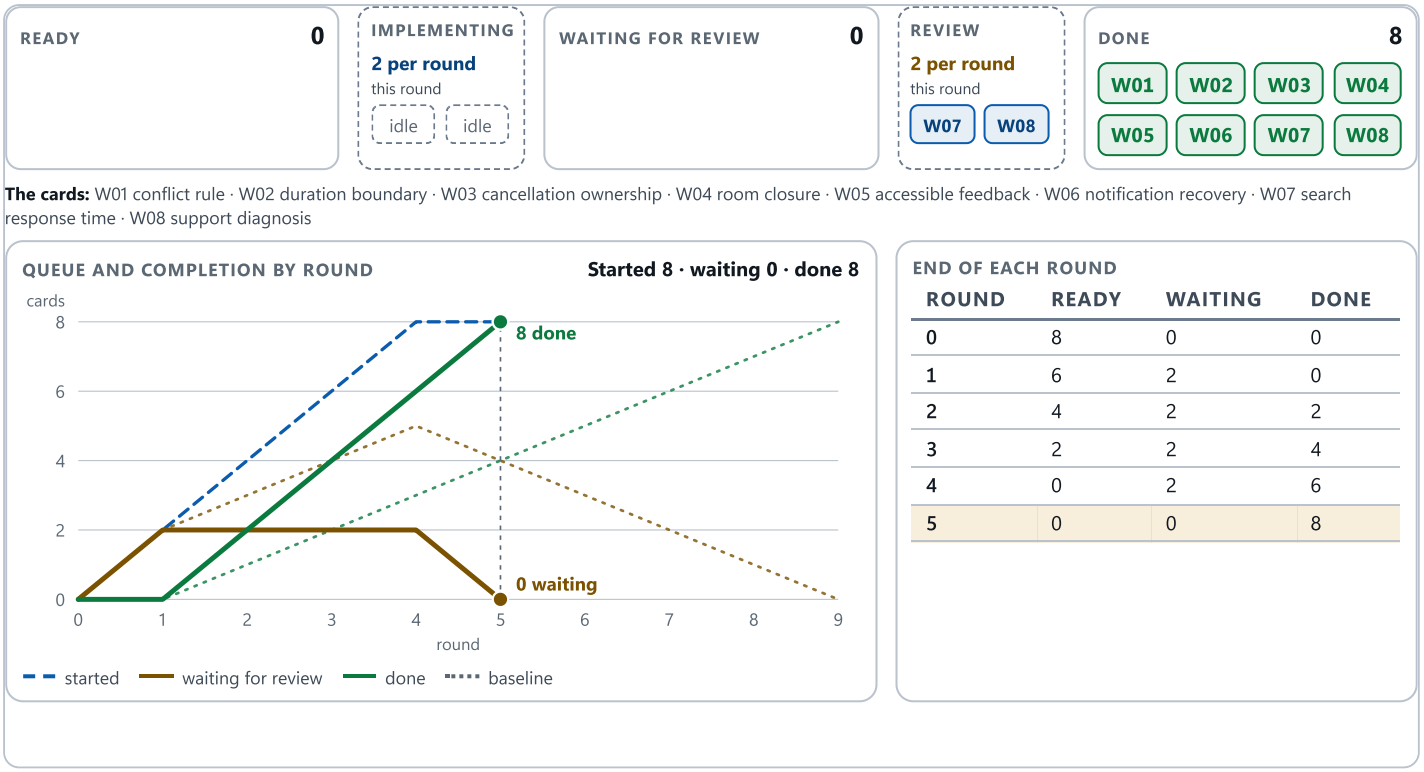
In short · Speed up the step that limits flow.

Make one stage faster: does the work finish sooner?

Panel 4 of 4 · Review doubled: done in round 5, and implementation now limits

Condition: **Changed: implementation 2, review 2 per round** State: **6 of 6**

Done in round 5; implementation limits.



Implementation 2 per round, Review 2 per round. Done in round 5; the queue never holds more than 2. With review relieved, implementation at 2 per round sets the pace: the bottleneck moved.

Principle · Local activity is not the same as system throughput; improve the limiting flow, not merely the busiest-looking stage.

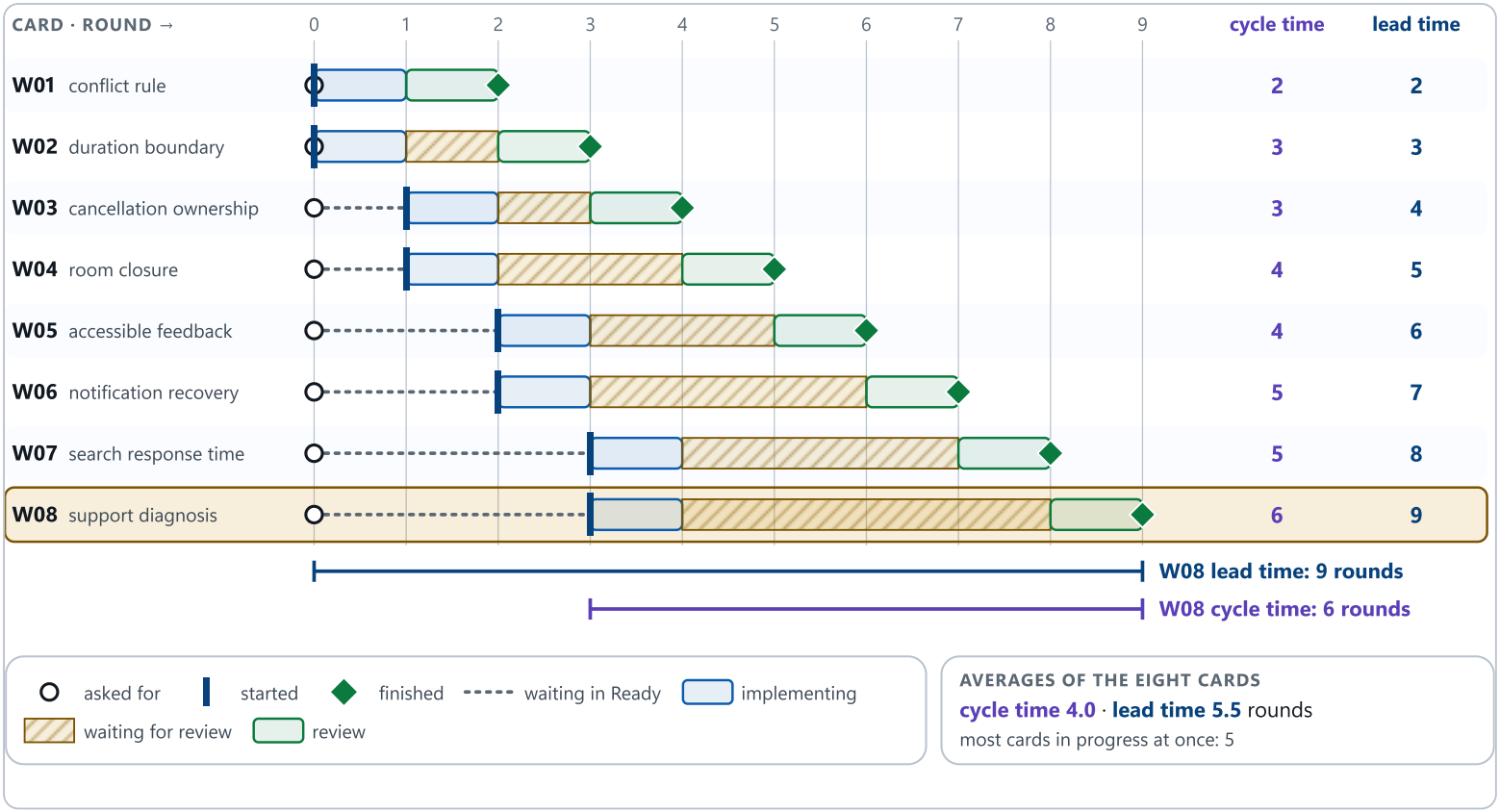
In short · Speed up the step that limits flow.

Which clock does the person who asked experience?

Panel 1 of 2 · Three markers per card, and both clocks with the waiting in Ready kept in view

Condition: **Baseline: no limit on work in progress** State: **5 of 5**

W08 waited 3 rounds in Ready: inside its lead time, outside its cycle time. On average, cycle time is 4.0 and lead time 5.5 rounds.



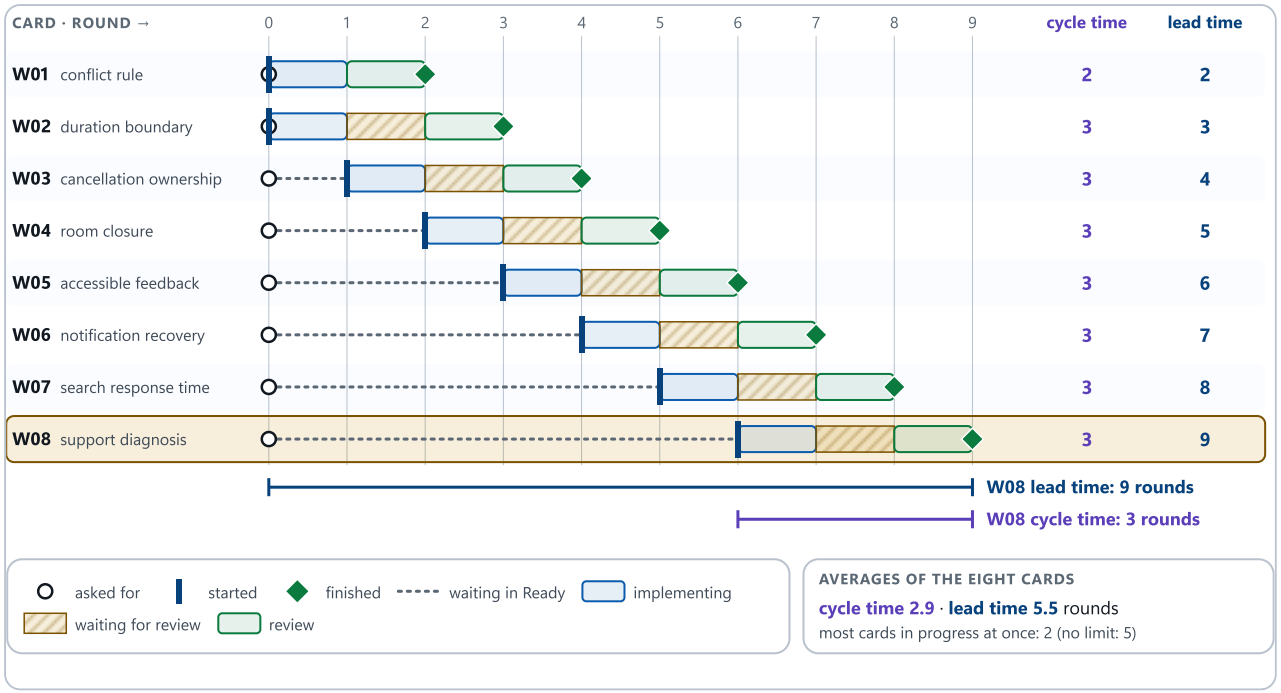
Markers: the request: every card is asked for at time 0; start is the declared start boundary: the beginning of the round in which the card is implemented; finish is the end of the round in which the card is reviewed. Lead time runs from the request (arrival) to completion; cycle time from the declared start to completion. W08: requested at 0, started at 3, finished at 9: lead time 9, cycle time 6, 3 rounds in Ready and 4 waiting for review. Averages: cycle 4.0, lead 5.5 rounds; at most 5 cards in progress.

Which clock does the person who asked experience?

Panel 2 of 2 · A work-in-progress limit of 2: shorter cycle time, the same lead time

Condition: **Changed: work-in-progress limit of 2** State: **5 of 5**

Every card still finishes in the same round.



Same eight cards, same capacities. With at most 2 cards in progress, W08 starts at 6: cycle time 3, lead time 9. Average cycle time 4.0 → 2.9 rounds; average lead time 5.5 → 5.5. At the end of round 6 the oldest started card is 2 rounds old (no limit: 4), while W08 has waited 6 rounds in Ready. The limit moved waiting from inside the cycle-time boundary to the Ready queue; every card finishes in the same round. S05 defines work in progress, throughput, work item age and cycle time; lead time is the stakeholder clock this lecture adds.

Principle · Timing metrics are meaningful only when their boundaries and excluded waiting are explicit.

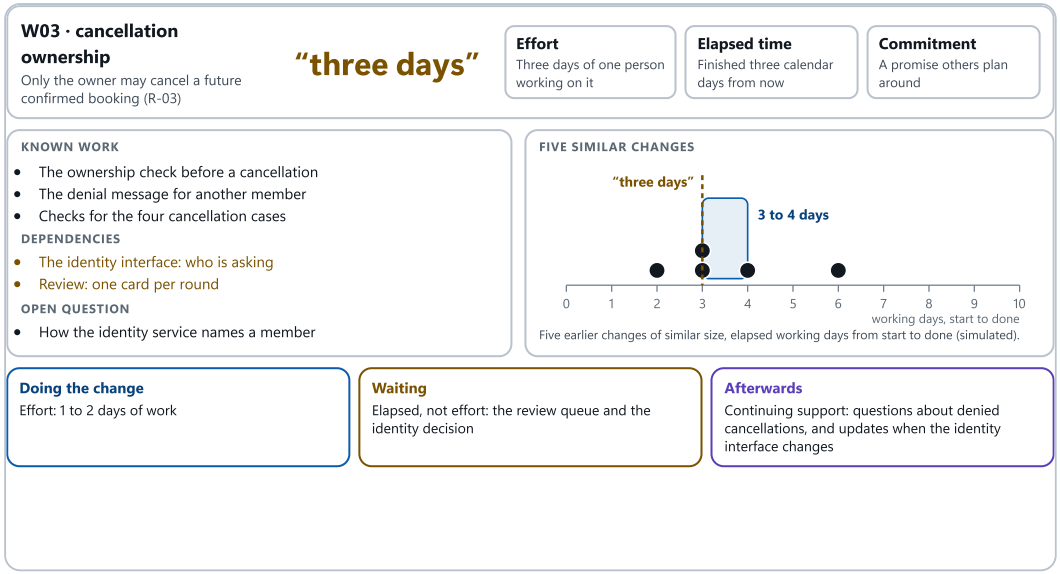
In short · Say where each clock starts and stops.

“Three days”: effort, elapsed time or a promise?

Panel 1 of 2 · The estimate with its meaning, its basis and what it leaves out

Condition: **Baseline: the identity interface is agreed** State: **6 of 6**

The cost continues after the estimate ends.



“Three days” can mean effort (three days of one person working on it), elapsed time (finished three calendar days from now) and commitment (a promise others plan around). Known work: the ownership check before a cancellation, the denial message for another member and checks for the four cancellation cases. Dependencies: the identity interface: who is asking and review: one card per round. Five similar changes took 2, 3, 3, 4, 6 days (simulated); three of the five similar changes took 3 to 4 days; one took 2 and one took 6, when review was busy. Estimate: 3 to 4 days elapsed, 1 to 2 days of effort, assuming the identity interface is agreed. The range summarizes the examples; it is not a statistical confidence interval. Cost after the estimate: doing the change: effort: 1 to 2 days of work; waiting: elapsed, not effort: the review queue and the identity decision; afterwards: continuing support: questions about denied cancellations, and updates when the identity interface changes.

Principle · An estimate is a conditional statement whose assumptions should travel with it.

In short · State what the number assumes.

“Three days”: effort, elapsed time or a promise?

Panel 2 of 2 · One assumption changes: the estimate widens, or carries its condition

Condition: **Changed: the identity interface is not agreed** State: **5 of 5**

Revise it when a condition changes.

W03 · cancellation ownership
Only the owner may cancel a future confirmed booking (R-03)

“three days”

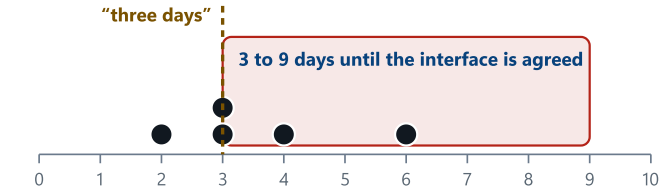
Effort	Elapsed time	Commitment
Three days of one person working on it	Finished three calendar days from now	A promise others plan around

KNOWN WORK
- The ownership check before a cancellation
- The denial message for another member
- Checks for the four cancellation cases

DEPENDENCIES
- The identity interface: who is asking
- Review: one card per round

OPEN QUESTION
- How the identity service names a member

FIVE SIMILAR CHANGES



Five earlier changes of similar size, elapsed working days from start to done (simulated).

THE ESTIMATE, WITH ITS CONDITION
3 to 4 days after the identity interface is agreed

WORTH DOING FIRST
A half-day conversation with the identity team: it narrows 3 to 9 days back to 3 to 4.

REVISE IT WHEN
The identity interface is agreed, or when review falls behind.

The identity interface is not yet agreed: the range widens to 3 to 9 days, or the estimate is qualified as 3 to 4 days after the identity interface is agreed. A half-day conversation with the identity team narrows it back. Revise the estimate when the identity interface is agreed, or when review falls behind.

Principle · An estimate is a conditional statement whose assumptions should travel with it.

In short · State what the number assumes.

What should the team learn before committing more work?

Panel 1 of 2 · The decision table and the baseline decision, with its opportunity cost

Condition: **Baseline: the identity choice is temporary** State: **6 of 6**

First: show that simultaneous bookings cannot overlap. The visual refinement waits: its value is the **opportunity cost**.

Option	Impact	Uncertainty	Reversibility	Cost of delay	Information gained	What it clarifies	If it goes wrong
Visual refinement	●○○ low	●○○ low	●●● high	●○○ low	●○○ little	How the booking page looks; nothing about the rules	Small: easy to change later
Show that simultaneous bookings cannot overlap	●●● high	●●● high	●●○ medium	●●● high	●●● large	Whether R-01 holds when two requests for one room arrive at the same moment	Two confirmed bookings for one room and time, found by the people who arrive
Investigate the unclear identity dependency	●●● high	●●● high	●○○ low if wrong	●●○ medium	●●● large	Who is asking, and how the identity service names a member (R-03 depends on it)	A wrong identity reference built into the cancellation rule

EVERY STEP CAN BE CHOSEN FOUR WAYS

investigate	A short, bounded look before committing
implement	Build now on the current assumptions
reduce scope	Ship owner cancellation first; staff actions later
defer	Leave the visual refinement until the rules are settled

THE DECISION

First: Show that simultaneous bookings cannot overlap. **Then:** Investigate the unclear identity dependency. **Deferred:** Visual refinement.

The highest cost of delay and a large information gain; the identity choice can still be corrected.

Opportunity cost: the value of the work the chosen step displaces: the visual refinement waits.

Criteria: impact, uncertainty, reversibility, cost of delay, information gained. Ordinal judgments with notes; no invented probabilities. Visual refinement: clarifies how the booking page looks; nothing about the rules; if it goes wrong, small: easy to change later. Show that simultaneous bookings cannot overlap: clarifies whether R-01 holds when two requests for one room arrive at the same moment; if it goes wrong, two confirmed bookings for one room and time, found by the people who arrive. Investigate the unclear identity dependency: clarifies who is asking, and how the identity service names a member (R-03 depends on it); if it goes wrong, a wrong identity reference built into the cancellation rule. Alternatives: investigate (a short, bounded look before committing); implement (build now on the current assumptions); reduce scope (ship owner cancellation first; staff actions later); defer (leave the visual refinement until the rules are settled). Decision: first show that simultaneous bookings cannot overlap, then investigate the unclear identity dependency; visual refinement deferred, because of the highest cost of delay and a large information gain; the identity choice can still be corrected. Opportunity cost: the value of the work the chosen step displaces: the visual refinement waits.

What should the team learn before committing more work?

Panel 2 of 2 · Only reversibility changes: the identity question moves first

Condition: **Changed: the choice becomes a data migration** State: **5 of 5**

Still uncertain until it reports.

Option	Impact	Uncertainty	Reversibility	Cost of delay	Information gained	What it clarifies	If it goes wrong
Visual refinement	●○○ low	●○○ low	●●● high	●○○ low	●○○ little	How the booking page looks; nothing about the rules	Small: easy to change later
Show that simultaneous bookings cannot overlap	●●● high	●●● high	●●○ medium	●●● high	●●● large	Whether R-01 holds when two requests for one room arrive at the same moment	Two confirmed bookings for one room and time, found by the people who arrive
Investigate the unclear identity dependency	●●● high	●●● high	○○○ very low	●●○ medium	●●● large	Who is asking, and how the identity service names a member (R-03 depends on it)	A wrong identity reference built into the cancellation rule

THE COST OVER ITS LIFE: A QUICK IDENTITY WORKAROUND NOW

Effort	Smallest
Elapsed waiting	None
Continuing support	High
Exit cost	High: a migration of every booking record
Opportunity cost	Low now

THE DECISION

First: Investigate the unclear identity dependency. **Then:** Show that simultaneous bookings cannot overlap. **Deferred:** Visual refinement.

A hard-to-reverse choice deserves more evidence before work builds on it.

Still uncertain: how the identity service names a member, until the investigation reports.

A temporary interface choice becomes a difficult data migration: the reversibility of the identity decision becomes very low: changing it means migrating every booking record. A hard-to-reverse choice deserves more evidence before work builds on it: first investigate the unclear identity dependency, then show that simultaneous bookings cannot overlap. A quick identity workaround now: effort smallest, elapsed waiting none, continuing support high, exit cost high: a migration of every booking record, opportunity cost low now; the smallest implementation estimate can be the most expensive choice over the change's life. Still uncertain: how the identity service names a member, until the investigation reports.

Principle · Prioritize not only output, but also the decisions and uncertainties that could make later work wasteful.

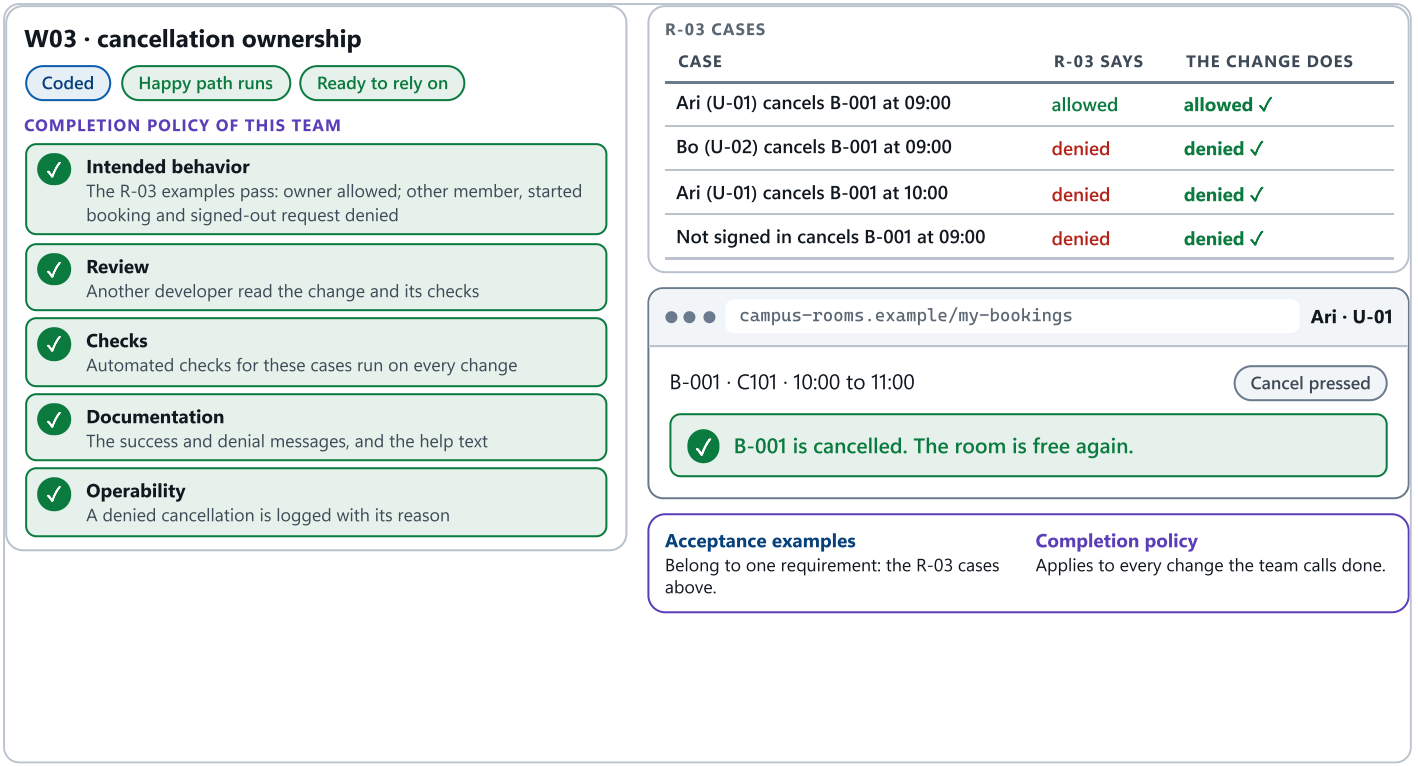
In short · Learn what could waste later work.

The code runs. Is the cancellation change done?

Panel 1 of 2 · The cancellation change: coded is not yet done

Condition: **Baseline: a change to cancellation behavior** State: **6 of 6**

R-03's examples accept one requirement. The completion policy applies to every change: in Scrum, the **Definition of Done**.



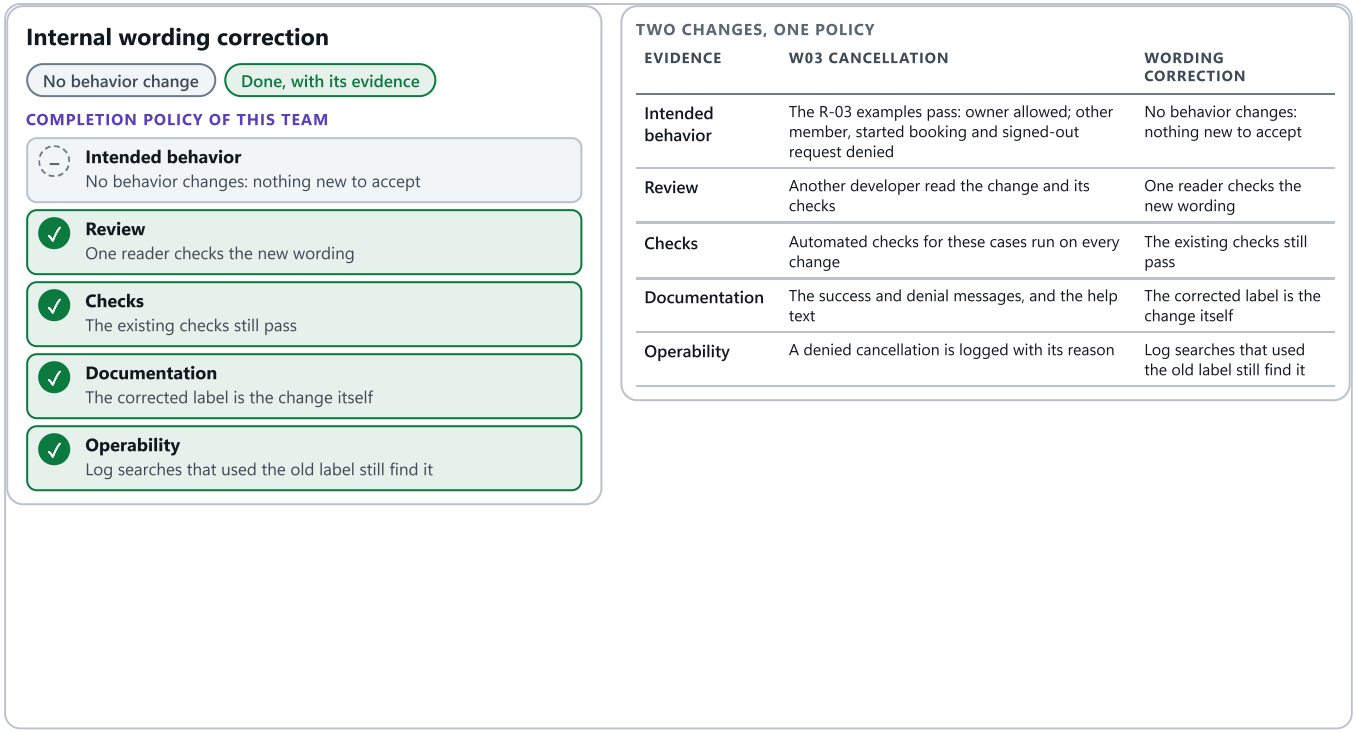
The happy path runs: Ari (U-01) cancels B-001 at 09:00: allowed. Missing at first: ownership check (Bo (U-02) cancels B-001: the code allows it, R-03 denies it); user feedback (after Cancel the screen shows no status message (R-05)). The team's completion policy: Intended behavior: the R-03 examples pass: owner allowed; other member, started booking and signed-out request denied; Review: another developer read the change and its checks; Checks: automated checks for these cases run on every change; Documentation: the success and denial messages, and the help text; Operability: a denied cancellation is logged with its reason. The R-03 examples accept one requirement; the completion policy applies to every change the team calls done (in Scrum: the Definition of Done). The Scrum Guide (S04) defines the Definition of Done as “A formal description of the state of the Increment when it meets the quality measures required for the product.”

The code runs. Is the cancellation change done?

Panel 2 of 2 · An internal wording correction: the same policy, the relevant evidence

Condition: **Changed: an internal wording correction** State: **4 of 4**

One policy; the evidence fits each change.



Internal wording correction: a typo fixed in an internal log label; no behavioral effect. Intended behavior: no behavior changes: nothing new to accept; Review: one reader checks the new wording; Checks: the existing checks still pass; Documentation: the corrected label is the change itself; Operability: log searches that used the old label still find it. The evidence is proportionate to the change, not an identical ceremony for every change.

Principle · A shared completion policy should make quality expectations visible and proportionate.

In short · Agree what done means, sized to the risk.

The review queue keeps growing. What should change?

Panel 1 of 2 · A process hypothesis: observation, change, expected effect and what would contradict it

Condition: **Baseline: share the review work** State: **6 of 6**

After the change: done in **round 5**, queue at most 2, issues found after review unchanged. The evidence agrees.

1 · OBSERVATION

The review queue grows: five cards wait at the end of round 4.



2 · PROPOSED CHANGE

Share review work: a second developer reviews, so review takes two cards per round.

3 · EXPECTED EFFECT

The review queue stays at two or fewer, and all eight cards are done in round 5.

4 · IT IS WRONG IF

The queue still grows, or reviewed cards come back with problems.

Side effect to watch: issues missed in review.

TALK ABOUT THE SYSTEM, NOT A PERSON

~~"The reviewer is too slow."~~

System: Two cards per round arrive at a review step that finishes one.

WHY BLAMELESS

People report delays and mistakes honestly only where reporting them is safe; blameless discussion and psychological safety make the process data trustworthy.

MEASURES, BEFORE AND AFTER

MEASURE	BEFORE	AFTER: SHARED REVIEW
Round in which all eight are done	9	5
Longest review queue	5	2
Issues found after review	1	1

Observation: the review queue grows: five cards wait at the end of round 4. Not "the reviewer is too slow", but: two cards per round arrive at a review step that finishes one. People report delays and mistakes honestly only where reporting them is safe; blameless discussion and psychological safety make the process data trustworthy. Proposal: share review work: a second developer reviews, so review takes two cards per round. Expected: the review queue stays at two or fewer, and all eight cards are done in round 5. It is wrong if the queue still grows, or reviewed cards come back with problems; side effect to watch: issues missed in review. Before: done in round 9, queue at most 5, 1 issue found after review. After: round 5, 2, 1. Simulated teaching values.

The review queue keeps growing. What should change?

Panel 2 of 2 · Faster but shallower: the side effect included

Condition: **Changed: faster review, more missed issues** State: **4 of 4**

Judge flow and quality together.

1 · OBSERVATION

The review queue grows: five cards wait at the end of round 4.



2 · PROPOSED CHANGE

Share review work: a second developer reviews, so review takes two cards per round.

3 · EXPECTED EFFECT

The review queue stays at two or fewer, and all eight cards are done in round 5.

4 · IT IS WRONG IF

The queue still grows, or reviewed cards come back with problems.

Side effect to watch: issues missed in review.

TALK ABOUT THE SYSTEM, NOT A PERSON

~~"The reviewer is too slow."~~

System: Two cards per round arrive at a review step that finishes one.

WHY BLAMELESS

People report delays and mistakes honestly only where reporting them is safe; blameless discussion and psychological safety make the process data trustworthy.

MEASURES, BEFORE AND AFTER

MEASURE	BEFORE	AFTER: SHARED REVIEW	AFTER: FASTER, SHALLOWER
Round in which all eight are done	9	5	5
Longest review queue	5	2	2
Issues found after review	1	1	4
All eight done, counting rework	9	5	7

A BALANCED EXPERIMENT

Judge the change on flow and quality together, not on one target number. When the results disagree with the expected effect, change the explanation, then the process.

Faster, shallower review: done in round 5, queue at most 2, but 4 issues found after review; four cards reopened after review, so the last card is done in round 7. Judge the change on flow and quality together, not on one target number.

Principle · Change a process deliberately, observe its consequences, and revise the explanation when the evidence disagrees.

In short · Try a change, watch its effects, revise.

Five measures of one board, and where each starts and stops

Each measure has two boundaries, shown on one card's timeline: W08 in the baseline, asked for at 0, started at 3, finished at 9.

W08 waited 3 rounds in Ready and 4 rounds for review.

MEASURE	BOUNDARIES	ON THIS BOARD	SOURCE
Work in progress	Items started but not finished, counted at a moment.	5 at the end of round 4 (baseline); at most 2 with the limit	Kanban Guide (S05)
Throughput	Items finished per unit of time, counted at the finish line.	1 card per round once review is busy (baseline)	Kanban Guide (S05)
Work item age	For an item not yet finished: from its start to now.	W08 at the end of round 6: 3 rounds (baseline)	Kanban Guide (S05)
Cycle time	From the declared start to the finish.	W08: 6 rounds (baseline), 3 with the limit	Kanban Guide (S05)
Lead time	From the request to the finish: the clock of the person who asked.	W08: 9 rounds, with or without the limit	the stakeholder clock of this lecture

A timing measure is only meaningful with its two boundaries, and with the waiting it leaves out named. S05 defines work in progress, throughput, work item age and cycle time; lead time is the stakeholder clock this lecture adds. Rounds are teaching units.

Eight principles, six checks

One principle per scene, then six checks: decide your answer before you open each one.

Principles

1 · ANCHOR

Shorter feedback loops can expose wrong assumptions earlier; useful increments must still make sense as a system.

In short: Show slices early; make them fit.

`week-04.html#/feedback-batches`

2 · BRIDGE

Choose coordination practices for the uncertainty and constraints of the work, not for their labels.

In short: Pick practices for the work, not the name.

`week-04.html#/methods-not-rituals`

3 · ANCHOR

Local activity is not the same as system throughput; improve the limiting flow, not merely the busiest-looking stage.

In short: Speed up the step that limits flow.

`week-04.html#/queue-and-bottleneck`

4 · BRIDGE

Timing metrics are meaningful only when their boundaries and excluded waiting are explicit.

In short: Say where each clock starts and stops.

`week-04.html#/wip-and-time`

5 · BRIDGE

An estimate is a conditional statement whose assumptions should travel with it.

In short: State what the number assumes.

`week-04.html#/estimates-with-assumptions`

6 · ANCHOR

Prioritize not only output, but also the decisions and uncertainties that could make later work wasteful.

In short: Learn what could waste later work.

`week-04.html#/risk-and-learning`

7 · BRIDGE

A shared completion policy should make quality expectations visible and proportionate.

In short: Agree what done means, sized to the risk.

`week-04.html#/definition-of-done`

8 · BRIDGE

Change a process deliberately, observe its consequences, and revise the explanation when the evidence disagrees.

In short: Try a change, watch its effects, revise.

`week-04.html#/retrospective-as-test`

Checks: decide your answer before reading the one below each question

1. Review processes one item per round. Will doubling implementation necessarily double completion?

> No. Review remains the limiting stage in the stated model.

2. What is the difference between iteration and increment?

> Iteration revisits or refines understanding or a solution; an increment adds usable capability.

3. Why might a risky assumption deserve attention before an easy visible feature?

> Resolving it can prevent later work from relying on a false premise.

4. What should accompany a process improvement claim?

> A baseline, a stated change, a relevant outcome measure, and attention to quality or other side effects.

5. Why can the smallest implementation estimate still be the more expensive choice?

> Waiting, integration, continuing support, exit cost and displaced valuable work can change the lifecycle comparison.

6. A library team writes changes faster, but maintainers can review no more than before. What should it inspect next?

> Inspect waiting work, review capacity, accepted changes and rework. Faster drafting can grow the queue without improving completion; choose the next improvement from the constraint and its quality consequences.

What carries forward, and where to read more

Next week asks how the design of the code decides what one change costs, and how safe it is.

Week 5 starts from this question:

When one requirement changes, which parts should need to change?

Carried forward to Week 5

- One prioritized change: changing a booking policy without causing widespread edits.
- The board and its measures: where work waits, and which clock a person experiences.
- Open decisions: the identity interface (investigate the unclear identity dependency), and room closures, still an open need (Week 14).

In practice · examples from 2026

Boards and queues live in tools such as Jira, GitHub Projects, Linear or Trello, whose reports chart the same work-in-progress, cycle-time and throughput measures this lecture builds by hand.

Sources for this lecture

S01 IEEE Computer Society, Guide to the Software Engineering Body of Knowledge (SWEBOK V4.0a) https://www.computer.org/education/bodies-of-knowledge/software-engineering

S02 ACM / IEEE-CS / AAAI, Computer Science Curricula 2023, final report https://csed.acm.org/final-report/

S03 Manifesto for Agile Software Development, principles https://agilemanifesto.org/principles.html

S04 Ken Schwaber and Jeff Sutherland, The Scrum Guide (November 2020) https://scrumguides.org/scrum-guide.html

S05 The Kanban Guide (May 2025 edition) https://kanbanguides.org/the-kanban-guide/2025.5/

Terms used this week

Batch
Work delivered together at one time; feedback waits for the whole batch.

Increment
A piece of work that adds usable capability.

Iteration
A pass that revisits or refines understanding or a solution.

Feedback loop
Something usable inspected by the people it serves, while a decision can still change.

Bottleneck
The stage that limits how much the whole system finishes; relieve it and the limit can move.

Work in progress (WIP)
Items started but not finished.

WIP limit
An explicit limit on items in progress: it controls admission, not demand.

Throughput
Items finished per unit of time.

Work item age
For an unfinished item, the time since it started.

Cycle time
From the declared start to the finish.

Lead time
From the request to the finish: the clock of the person who asked.

Estimate
A conditional statement: a range with its basis, assumptions and the condition for revising it.

Cost of delay
What is lost while a step waits.

Reversibility
How costly a decision is to change later.

Opportunity cost
The value of the work a chosen step displaces.

Completion policy
What a team agrees “done” needs, in proportion to the risk; in Scrum, the Definition of Done.

Blameless review
Discussing the system rather than a person, so problems are reported honestly.